

ESTUDO DIRIGIDO

Refatore o projeto aplicando princípios de projeto e boas práticas de orientação a objetos, **sem alterar o comportamento esperado da aplicação**.

Analise o código e identifique onde cada um dos princípios abaixo pode ser aplicado.

Apresentação do código

O projeto representa uma aplicação de pedidos, responsável por organizar informações de clientes, produtos, formas de pagamento, descontos e opções de entrega.

O código possui classes relacionadas ao cadastro de clientes, gerenciamento de produtos e cálculo do valor total dos pedidos. Também apresenta diferentes formas de pagamento, como cartão e boleto, além de regras específicas para aplicação de descontos e escolha do tipo de entrega.

O objetivo da aplicação é permitir que um pedido seja criado, tenha seus produtos adicionados, descontos sejam calculados, o pagamento seja processado e a entrega seja definida. Cada parte do sistema possui comportamentos próprios, mas algumas responsabilidades podem estar concentradas nas mesmas classes ou depender diretamente de implementações específicas.

Durante a atividade, o código deverá ser analisado para identificar problemas de organização, acoplamento, duplicação e uso inadequado de herança ou interfaces. A refatoração deve preservar o funcionamento da aplicação, mas tornar sua estrutura mais clara, flexível e preparada para receber novos tipos de produtos, pagamentos, descontos e entregas sem exigir alterações excessivas nas classes existentes.

Princípios que devem ser aplicados

1. Princípio da Responsabilidade Única — SRP

Uma classe deve possuir uma responsabilidade principal bem definida.

Identifique classes ou métodos que estejam realizando tarefas demais e reorganize essas responsabilidades.

2. Princípio da Segregação de Interfaces — ISP

Uma classe não deve ser obrigada a depender de operações que não utiliza.

Analise as interfaces existentes e verifique se elas podem ser divididas em abstrações menores e mais específicas.

3. Composição sobre Herança

Utilize herança apenas quando existir uma verdadeira relação de especialização (“é um”).

Quando um objeto apenas precisar utilizar o comportamento de outro, considere utilizar **composição**.

4. Princípio do Menor Conhecimento — Lei de Demeter

Um objeto deve conhecer o mínimo possível sobre a estrutura interna de outros objetos.

Procure situações em que uma classe precise navegar por vários objetos para obter uma informação e reduza esse acoplamento.

5. Princípio Aberto/Fechado — OCP

O código deve estar **aberto para extensão, mas fechado para modificação**.

Identifique estruturas que precisariam ser alteradas sempre que surgisse um novo tipo de comportamento e refatore-as utilizando abstrações e polimorfismo.

6. Princípio da Inversão de Dependência — DIP

Classes de mais alto nível não devem depender diretamente de implementações concretas.

Utilize abstrações para permitir que diferentes comportamentos possam ser fornecidos ao sistema sem que a classe principal precise conhecer suas implementações.

7. Princípio da Substituição de Liskov — LSP

Uma classe derivada deve poder substituir sua classe base sem alterar o comportamento esperado do sistema.

Analise as relações de herança existentes e verifique se todas as subclasses realmente respeitam o contrato e as expectativas definidas pela abstração.

Orientações

Durante a refatoração:

- mantenha o comportamento atual da aplicação;
- execute o projeto após as alterações;
- evite duplicação desnecessária;
- procure reduzir o acoplamento entre as classes;
- utilize abstrações quando diferentes comportamentos puderem variar;
- mantenha cada classe com responsabilidades claras.